This will create a `Product` model, mapped to a `products` table at the database. By doing this you'll also have the ability to map the columns of each row in that table with the attributes of the instances of your model. Suppose that the `products` table was created using an SQL sentence like:

```
CREATE TABLE products (
   id int(11) NOT NULL auto_increment,
   name varchar(255),
   PRIMARY KEY (id)
);
```

Following the table schema above, you would be able to write code like the following:

```
p = Product.new
p.name = "Some Book"
puts p.name # "Some Book"
```

4 Overriding the Naming Conventions

What if you need to follow a different naming convention or need to use your Rails application with a legacy database? No problem, you can easily override the default conventions.

You can use the `ActiveRecord::Base.table_name=` method to specify the table name that should be used:

```
class Product < ActiveRecord::Base
  self.table_name = "PRODUCT"
end
```

If you do so, you will have to define manually the class name that is hosting the fixtures (class_name.yml) using the `set_fixture_class` method in your test definition:

```
class FunnyJoke < ActiveSupport::TestCase
  set_fixture_class funny_jokes: Joke
  fixtures :funny_jokes
  ...
end
```

It's also possible to override the column that should be used as the table's primary key using the `ActiveRecord::Base.primary_key=` method:

```
class Product < ActiveRecord::Base
  self.primary_key = "product_id"
end
```

5 CRUD: Reading and Writing Data

CRUD is an acronym for the four verbs we use to operate on data: **C**reate, **R**ead, **U**pdate and **D**elete. Active Record automatically creates methods to allow an application to read and manipulate data stored within its tables.

5.1 Create

Active Record objects can be created from a hash, a block or have their attributes manually set after creation. The `new` method will return a new object while `create` will return the object and save it to the database.

For example, given a model `User` with attributes of `name` and `occupation`, the `create` method call will create and save a new record into the database:

```
user = User.create(name: "David", occupation: "Code Artist")
```

Using the `new` method, an object can be instantiated without being saved:

```
user = User.new
user.name = "David"
user.occupation = "Code Artist"
```

A call to `user.save` will commit the record to the database.